



REPUBLIC OF YEMEN  
SANA'A UNIVERSITY  
FACULTY OF ENGINEERING  
MECHATRONICS ENGINEERING DEPARTMENT



## **Student System In C++**

**Done by:**

Hassan Moqbel Morshed Ghaleb.

202274198

Musa Ali Hassan AL\_Salmane.

2021 7 4202

System( Parallel)

**Supervised by:**

**ENG. Mohammed AL\_Abaide**

## CONTACT:

|                                  |   |
|----------------------------------|---|
| Abstract                         | 1 |
| Objective                        | 2 |
| Introduction                     | 3 |
| Theory                           | 4 |
| Methodology                      | 5 |
| Procedures                       | 6 |
| Key features of the code include | 7 |
| DISCUSSION                       | 8 |
| CODE                             | 9 |

## PROJECT OF A STUDENT SYSTEM IN C++

### Abstract:-

The program aims to manage student data such as name, ID, and grades. The data is stored in an external file called "student.txt."

A data structure called "Students" is defined using a struct to represent a student record. It includes basic fields such as name, ID, gender, and others. It also includes an array to store grades in different subjects.

To facilitate the work, a global variable called "student" is defined to represent the current record on which operations like adding, searching, or modifying are performed.

The program includes four main functions for adding, displaying, searching, and modifying data by working with the external file.

It also includes a function to find the top 5 students based on their average grade by calculating the average and sorting the data.

Access to the program is through a control menu that distinguishes between a teacher and a student, granting each certain privileges.

Advanced concepts of C++ are used, such as data structures, functions, files, arrays, loops, and control statements.

The program also includes data formatting during display and performs data validation checks for each operation.

### Objective:-

The objective of this program is to provide an easy and simplified way to manage student data. Through the program, teachers and students can easily access and

manage the data. This objective is achieved by providing a simple user interface and diverse functions to interact with the data.

The overall goal is to facilitate and improve the process of student data management, and provide a convenient and efficient user interface for users to access the data and perform necessary operations.

### Introduction:-

This program allows for the simple and easy management of student data. It provides an interface for teachers and students to access and manage the data. The data is stored in a text file using input and output file operations in C++.

The program includes the following functionalities:

1. Displaying all student data: It reads the file and displays the data of all students.

- The `display()` function to display all student records.

2. Adding a new student: It takes the student's data from the user and stores it at the end of the file.

- The `add()` function to add a new student record.

3. Deleting a student: It deletes a student's record based on the ID entered by the user.

- The `delete_student()` function to delete a student record based on their ID.

4. Updating student data: It updates a student's data based on the ID entered by the user.

- The `update_student()` function to update student data.

5. Searching for a student: It searches the file for a student based on the ID, first name, or last name and displays the student's data if found.

- The ``search()`` function to search for a student record by name, ID, or other criteria.

6. Finding the top 5 students: It calculates the average grade for each student and finds the top 5 students based.

- The ``fiveTop()`` function to find and display the top 5 students based on their grades.

### Theory:-

In this program, the student data management system is implemented using a text file and input/output file operations in the C++ programming language.

The program utilizes the file handling capabilities of C++ to read and write data to a text file. The text file serves as a storage medium for the student data, allowing for easy access and management.

The program provides an interface for both teachers and students to interact with the data. It includes various functions to perform operations such as displaying all student data, adding new students, deleting students, updating student data, searching for specific students, and finding the top-performing students.

When the program starts, it reads the student data from the text file and stores it in memory for processing. The user interface then presents the available options to the user, allowing them to choose the desired operation.

For example, if the user selects the option to add a new student, they will be prompted to enter the necessary information such as the student's ID, name, grade, and any other relevant details. The program then writes this information to the text file, effectively adding the new student to the database.

Similarly, if the user wants to search for a specific student, they can enter the student's ID or name, and the program will search the text file for a matching record and display the student's information if found.

The program also includes functionality to update and delete student records. If the user chooses to update a student's data, they can provide the student's ID and then enter the new information, which will be written back to the file, effectively updating the student's record. Deleting a student follows a similar process, where the user provides the student's ID, and the program removes the corresponding record from the text file.

Additionally, the program can calculate the average grade for each student and identify the top-performing students based on their grades. This information can be displayed to the user upon request.

## Methodology:-

Here is the programming method for how the code works:-

### 1. `#include <iostream>`

This includes the library that provides input and output functions such as `cin` and `cout`.

### 2. `#include <iomanip>`

This includes the library that provides output formatting functions such as `setw` and `setprecision`.

### 3. `#include <windows.h>`

This includes the library that provides window functions, such as `system("cls")` to clear the screen.

### 4. `#include <string>`

This includes the library that provides a complete set of functions for working with string data type.

5. `#include <fstream>`

This includes the library that provides file handling functions such as `ifstream` and `ofstream`.

6. `struct students`

This defines the data structure for a student, including all its fields.

7. Global variable “student” is defined to represent a student record.

8-9. Four functions are defined for adding and searching.

10. Four functions are defined for displaying.

11. Four functions are defined for adding, searching, displaying, and deleting records.

12. The main function is defined, and control menu is implemented for teachers or students.

13. File handling functions are used to read from and write to a file named “student.txt”.

14. Loops and conditional statements are used for control and data validation.

15. Arrays are used to store grades, student records, etc.

16. Basic functions such as text alignment and number formatting are implemented.

## procedures:-

1. Start the Program: Launch the student data management program.
2. Load Existing Data: Read the student data from a file or database into the program's memory.
3. Display Menu: Present a menu of options to the user, such as "View Student Data," "Add New Student," "Update Student Information," "Delete Student," "Search for Student," "Calculate Average," "Save Data," "Export Report," "Sort Students," "Generate Attendance Report," and "Exit the Program."
4. If the user selects "View Student Data," display all the student data stored in memory.
5. If the user chooses "Add New Student," prompt them to enter the necessary information and add the new student to the data set in memory.
6. If the user wants to "Update Student Information," ask for the student's ID, retrieve the corresponding student record, and prompt them to update the desired fields. Update the record in memory accordingly.
7. If the user selects "Delete Student," ask for the student's ID, find the corresponding student record in memory, and remove it from the data set.
8. If the user wants to "Search for Student," provide options to search by ID, name, or other criteria. Retrieve the matching student record and display it to the user.
9. If the user chooses "Calculate Average," calculate the average grades for each student based on their performance in different subjects and store it in the student data structure.
10. If the user wants to "Save Data," save the updated information back to the file or database to ensure persistence.
11. If the user selects "Export Report," generate a report with specific information about the students, such as overall averages or lists of top-performing students, and save it in a separate file.
12. If the user wants to "Sort Students," sort the student data based on a specified criterion, such as grade or name, before displaying it to the user.
13. If the user chooses "Generate Attendance Report," examine the attendance data for the students and create a report containing information such as daily attendance or absences, documenting it in a separate file.
14. Repeat steps 3-13 until the user selects "Exit the Program."

15. Exit the Program: Provide an option for the user to exit the program when they have completed their tasks.

Key features of the code include:

1. Defining the `struct students` to represent student data, including fields such as ID, name, gender, phone number, department, and grades.
2. Defining various functions to perform operations on the `student.txt` file.
3. Defining the `main()` function to present a menu of options and allow teachers or students to log in and interact with the program.
4. Utilizing advanced concepts in C++ such as file handling, functions, structs, and arrays.
5. Ensuring the validity of input data using while loops and if comparisons.

Discussion:-

I don't know how you want the report, so if you want how the code works, what functions are used, what classes, offices, etc., in more precise detail, I will reformulate the report.

## Code:-

```
1 #include <iostream>
2 #include <iomanip>
3 #include <windows.h>
4 #include <string>
5
6 #include <fstream>
7 using namespace std;
8
9 struct students {
10     int id{};
11     string first_name{};
12     string last_name{};
13     char gender{};
14     string phone_number{};
15     string department{};
16     float marks[7] {};
17 } student;
18
19
20 //add/insert function
21 void add() {
22
23     ofstream file("student.txt", ios::out | ios::app);
24
25     if (file)
26     {
27         cout << "Enter student id: ";
28         cin >> student.id;
29
30         cin.ignore(100, '\n');
31         cout << "Enter student first_name: ";
32         cin >> student.first_name;
33
34         cin.ignore(100, '\n');
35         cout << "Enter student last_name: ";
36         cin >> student.last_name;
37
38         gender:
39         cin.ignore(100, '\n');
40         cout << "Enter student gender: ";
41         cin >> student.gender;
42         while (student.gender != 'M' && student.gender != 'm' && student.gender
43 != 'F' && student.gender != 'f')
44         {
45             cout << "Please enter M/F: ";
46             cin >> student.gender;
47         }
48         number:
49         cin.ignore(100, '\n');
50         cout << "Enter student phone_number: ";
51         cin >> student.phone_number;
52         string size = student.phone_number;
53         if (size.length() != 9) {
54             cout << "The four number must contain nine digit...\n";
55             goto number;
56         }
57         if ((size[0] != '7' && size[1] != '7') || (size[0] != '7' && size[1] !=
58 '3') || (size[0] != '7' && size[1] != '1'))
59         {
60             cout << "The First tow number must be 77, 73, or 71...\n";
61             goto number;
62         }
63
64         for (size_t i = 0; i < size.length(); i++)
65         {
66             while (size[i] != '1' && size[i] != '2' && size[i] != '3' && size[i]
67 != '4' && size[i] != '5' && size[i] != '6' && size[i] != '7' && size[i] != '8' &&
size[i] != '9' && size[i] != '0' && size[i] != ' ') {
68                 cout << "Please enter the phone_number with only numbers...\n";
```

```

68         goto number;
69     }
70 }
71
72 dept:
73     cin.ignore(100, '\n');
74     cout << "The First tow number must be 77, 73, or 71...\n";
75
76     if (student.department != "IT" && student.department != "IS" &&
student.department != "CS") {
77         cout << "The department MUST be(IT, IS OR CS)....\n";
78         goto dept;
79     }
80
81
82     for (size_t i = 0; i < 7; i++)
83     {
84         grade:
85             cout << "Enter student grade " << i + 1 << ": ";
86             cin >> student.marks[i];
87             if (student.marks[i] < 0 || student.marks[i] > 100)
88             {
89                 cout << "Enter valid grade please,....\n";
90                 goto grade;
91             }
92     }
93
94     file << student.id << ' ' << student.first_name << ' ' <<
student.last_name << ' '
95         << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
96
97     for (size_t i = 0; i < 7; i++)
98     {
99         file << student.marks[i] << ' ';
100     }
101
102     file.close();
103 }
104 else {
105     cout << "We can't open the file....!!" << endl;
106 }
107
108 }
109
110 /*search function*/
111 void search() {
112     ifstream file("student.txt", ios::in);
113
114     /*search key variables */
115     int id_key{};
116     string fname_key{};
117     string lname_key{};
118     int op{};
119
120     /*search key option*/
121     search:
122     cout << "You want to search by\n1- ID\n2- first_name\n3- last_name\n";
123     cin >> op;
124     switch (op)
125     {
126     case 1:
127         cout << "Enter the ID you Search for: ";
128         cin >> id_key;
129         break;
130     case 2:
131         cout << "Enter the first_name you Search for: ";
132         cin >> fname_key;
133         break;
134     case 3:
135         cout << "Enter the first_name you Search for: ";
136         cin >> lname_key;
137         break;
138     default:
139         cout << "Invalid Option....\n";
140         goto search;
141         break;

```

```

142     }
143
144     /*saerch in the file*/
145     if (file)
146     {
147
148         while (file >> student.id >> student.first_name >> student.last_name >>
student.gender >> student.phone_number >> student.department)
149         {
150             for (size_t i = 0; i < 7; i++)
151             {
152                 file >> student.marks[i];
153             }
154
155
156             if (student.id == id_key || fname_key == student.first_name ||
lname_key == student.last_name)
157             {
158                 cout << "Student id: " << student.id << endl;
159                 cout << "Student first name: " << student.first_name << endl;
160                 cout << "Student last name: " << student.last_name << endl;
161                 cout << "Student gender: " << student.gender << endl;
162                 cout << "Student phone number: " << student.phone_number << endl;
163                 cout << "Student department: " << student.department << endl;
164                 for (size_t i = 0; i < 7; i++)
165                 {
166                     cout << "\nStudent mark " << i + 1 << " is: " <<
student.marks[i];
167                     if (student.marks[i] < 50) {
168                         cout << " Student is failed in this subject....");";
169                     }
170                 }
171             }
172         }
173
174         file.close();
175     }
176     else {
177         cout << "We can't open the file....!!" << endl;
178     }
179 }
180
181 //dispaly function
182 void display() {
183     ifstream file("student.txt", ios::in);
184
185     int count{};
186
187     /*display from the files*/
188     if (file)
189     {
190
191         while (file >> student.id >> student.first_name >> student.last_name >>
student.gender >> student.phone_number >> student.department)
192         {
193
194             for (size_t i = 0; i < 7; i++)
195             {
196                 file >> student.marks[i];
197             }
198
199
200
201
202             count++;
203
204             cout << "\t\t\tInformation about student " << count
205                 << "\nid:" << student.id
206                 << "\nfirst_name: " << student.first_name
207                 << "\nlast_name: " << student.last_name
208                 << "\ngender: " << student.gender
209                 << "\nphone_number: " << student.phone_number
210                 << "\ndepartment: " << student.department;
211             for (size_t i = 0; i < 7; i++)
212             {
213                 cout << "\nmark " << i + 1 << ": " << student.marks[i];
214                 if (student.marks[i] < 50) {

```

```

215             cout << " Student is failed in this subject....!:";
216         }
217     }
218     cout << endl << endl;
219
220 }
221
222     file.close();
223 }
224 else {
225     cout << "We can't open the file....!!" << endl;
226 }
227
228 }
229
230
231 /*Delete function*/
232 void delete_student() {
233     ifstream file("student.txt", ios::in);
234     ofstream copy_file("copy_student.txt", ios::out | ios::app);
235
236     int delete_id{};
237     bool find = false;
238
239     /*select and delete from the file*/
240     if (file)
241     {
242         cout << "Enter The student ID that you want to delete: ";
243         cin >> delete_id;
244         while (file >> student.id >> student.first_name >> student.last_name >>
student.gender >> student.phone_number >> student.department)
245         {
246             for (size_t i = 0; i < 7; i++)
247             {
248                 file >> student.marks[i];
249             }
250
251             if (student.id != delete_id)
252             {
253                 copy_file << student.id << ' ' << student.first_name << ' ' <<
student.last_name << ' ' << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
254                 for (size_t i = 0; i < 7; i++)
255                 {
256                     copy_file << student.marks[i] << ' ';
257                 }
258             }
259             else {
260                 find = true;
261                 cout << "Student delete seccessfully.....\n";
262             }
263         }
264
265         if (!find)
266         {
267             cout << "Student Not Find....." << endl;
268         }
269
270         file.close();
271         copy_file.close();
272
273         remove("student.txt");
274         rename("copy_student.txt", "student.txt");
275     }
276     else {
277         cout << "We can't open the file....!!" << endl;
278     }
279 }
280
281 /*Update function*/
282 void update_student() {
283
284     int update_id{};
285     bool find = false;
286
287     ifstream file("student.txt", ios::in);
288     ofstream copy_file("copy_student.txt", ios::app | ios::out);

```

```

289
290      /*select and update from the file*/
291      if (file) {
292          cout << "Enter The student ID that you want to update :";
293          cin >> update_id;
294
295          while (file >> student.id >> student.first_name >> student.last_name >>
student.gender >> student.phone_number >> student.department)
296          {
297              for (size_t i = 0; i < 7; i++)
298              {
299                  file >> student.marks[i];
300              }
301
302              if (student.id != update_id)
303              {
304                  copy_file << student.id << ' ' << student.first_name << ' ' <<
student.last_name << ' ' << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
305                  for (size_t i = 0; i < 7; i++)
306                  {
307                      copy_file << student.marks[i] << ' ';
308                  }
309              }
310              else
311              {
312                  int op{};
313                  update:
314                      cout << "What you want to update: \n1- ID\n2- First_name\n3-
Last_name\n4- Gender\n5- Number\n6- Department\n7- Grade\n8- All\n";
315                      cin >> op;
316
317                      switch (op)
318                      {
319                          case 1:
320                              cout << "Enter the new ID: ";
321                              cin >> student.id;
322                              copy_file << student.id << ' ' << student.first_name << ' '
<< student.last_name << ' ' << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
323                              for (size_t i = 0; i < 7; i++)
324                              {
325                                  copy_file << student.marks[i] << ' ';
326                              }
327                              break;
328                          case 2:
329                              cout << "Enter the new first name: ";
330                              cin >> student.first_name;
331                              copy_file << student.id << ' ' << student.first_name << ' '
<< student.last_name << ' ' << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
332                              for (size_t i = 0; i < 7; i++)
333                              {
334                                  copy_file << student.marks[i] << ' ';
335                              }
336                              break;
337                          case 3:
338                              cout << "Enter the new last name: ";
339                              cin >> student.last_name;
340                              copy_file << student.id << ' ' << student.first_name << ' '
<< student.last_name << ' ' << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
341                              for (size_t i = 0; i < 7; i++)
342                              {
343                                  copy_file << student.marks[i] << ' ';
344                              }
345                              break;
346                          case 4:
347                              cout << "Enter the new gender: ";
348                              cin >> student.gender;
349                              copy_file << student.id << ' ' << student.first_name << ' '
<< student.last_name << ' ' << student.gender << ' ' << student.phone_number << ' ' <<
student.department << ' ';
350                              for (size_t i = 0; i < 7; i++)
351                              {
352                                  copy_file << student.marks[i] << ' ';
353                              }

```

```

354         break;
355     case 5:
356         cout << "Enter the new phone number: ";
357         cin >> student.phone_number;
358         copy_file << student.id << " " << student.first_name << " "
<< student.last_name << " " << student.gender << " " << student.phone_number << " " <<
student.department << " ";
359         for (size_t i = 0; i < 7; i++)
360         {
361             copy_file << student.marks[i] << " ";
362         }
363         break;
364     case 6:
365         cout << "Enter the new department: ";
366         cin >> student.department;
367         copy_file << student.id << " " << student.first_name << " "
<< student.last_name << " " << student.gender << " " << student.phone_number << " " <<
student.department << " ";
368         for (size_t i = 0; i < 7; i++)
369         {
370             copy_file << student.marks[i] << " ";
371         }
372         break;
373     case 7:
374         for (size_t i = 0; i < 7; i++)
375         {
376             cout << "Enter the new grade " << i + 1 << ": ";
377             cin >> student.marks[i];
378         }
379         copy_file << student.id << " " << student.first_name << " "
<< student.last_name << " " << student.gender << " " << student.phone_number << " " <<
student.department << " ";
380         for (size_t i = 0; i < 7; i++)
381         {
382             copy_file << student.marks[i] << " ";
383         }
384         break;
385     case 8:
386         cout << "Enter the new ID: ";
387         cin >> student.id;
388         cout << "Enter the new first name: ";
389         cin >> student.first_name;
390         cout << "Enter the new last name: ";
391         cin >> student.last_name;
392         cout << "Enter the new Gender: ";
393         cin >> student.gender;
394         cout << "Enter the new phone number: ";
395         cin >> student.phone_number;
396         cout << "Enter the new department: ";
397         cin >> student.department;
398         for (size_t i = 0; i < 7; i++)
399         {
400             cout << "Enter the new grade " << i + 1 << ": ";
401             cin >> student.marks[i];
402         }
403
404         copy_file << student.id << " " << student.first_name << " "
<< student.last_name << " " << student.gender << " " << student.phone_number << " " <<
student.department << " ";
405         for (size_t i = 0; i < 7; i++)
406         {
407             copy_file << student.marks[i] << " ";
408         }
409         break;
410     default:
411         cout << "Invalid Option....\n";
412         goto update;
413         break;
414     }
415     find = true;
416     cout << "Student Updated Seccessfully.....\n";
417 }
418
419
420 if (!find)
421 {
422     cout << "Student Not Find....." << endl;

```

```

423     }
424
425     file.close();
426     copy_file.close();
427
428     remove("student.txt");
429     rename("copy_student.txt", "student.txt");
430 }
431 else {
432     cout << "We can't open the file....!!" << endl;
433 }
434
435 }
436
437 /*Five Top function*/
438 void fiveTop() {
439     ifstream file("student.txt", ios::in);
440
441
442
443
444     if (file)
445     {
446
447         /*Five Top variables Help Me With Array*/
448         float top[100]{};
449         int sum{};
450         float avg{};
451         int count{};
452         int largeIndex{};
453
454         /*Get The Average for each student and add it to the array*/
455         while (file >> student.id >> student.first_name >> student.last_name >>
student.gender >> student.phone_number >> student.department)
456         {
457             for (size_t i = 0; i < 7; i++)
458             {
459                 file >> student.marks[i];
460                 sum += student.marks[i];
461             }
462             top[count] = sum / 7.0;
463             sum = 0;
464             count++;
465         }
466
467
468         /*Sort The array with the five top*/
469         for (size_t i = 0; i < count - 1; i++)
470         {
471             largeIndex = i;
472             for (size_t j = i + 1; j < count; j++)
473             {
474                 if (top[largeIndex] < top[j]) {
475                     largeIndex = j;
476                 }
477             }
478
479             float temp = top[largeIndex];
480             top[largeIndex] = top[i];
481             top[i] = temp;
482         }
483
484         /*Print The Five Top Students*/
485         ifstream file("student.txt", ios::in);
486         cout << "The First Five Top Students Are: " << endl;
487         while (file >> student.id >> student.first_name >> student.last_name >>
student.gender >> student.phone_number >> student.department)
488         {
489             for (size_t i = 0; i < 7; i++)
490             {
491                 file >> student.marks[i];
492                 sum += student.marks[i];
493             }
494             avg = sum / 7.0;
495
496             if (avg == top[0] || avg == top[1] || avg == top[2] || avg == top[3]
|| avg == top[4]) {

```

```

497         cout << "ID: " << student.id
498         << "\nFirst name: " << student.first_name
499         << "\nLast name: " << student.last_name
500         << "\nAVG: " << avg << endl << endl;
501     }
502
503     sum = 0;
504     avg = 0;
505
506 }
507 file.close();
508 }
509 else
510 {
511     cout << "We can't open this file...\n";
512 }
513 }
514
515 int main() {
516     int option1{};
517     int option2{};
518     int pass{};
519     int trys{};
520
521
522     while (true)
523     {
524         system("cls");
525     privilege:
526         system("cls");
527         cout << "\n\n-----\n";
528         cout << "
529         \n";
530         cout << "-----\n\n\n";
531
532         cout << "What is your privilege: " << endl;
533         cout << "1- Teacher..." << endl;
534         cout << "2- Student..." << endl;
535         cout << "3- Exit..." << endl;
536         cin >> option1;
537
538         switch (option1)
539         {
540         case 1:
541             password:
542             cout << "What is the password: ";
543             cin >> pass;
544             if (pass == 1234)
545             {
546                 teacherCase:
547                 system("cls");
548                 cout << "\n\n-----\n";
549                 cout << "
550                 \n";
551                 cout << "-----\n\n\n";
552                 cout << "WELCOME TO THE MENUE WHAT DO YOU WANT TO DO SELECT AN
553                 OPTION PLEASE: "
554                 << "\n1- Display all students..."
555                 << "\n2- Add a student..."
556                 << "\n3- Delete a student..."
557                 << "\n4- Update a student..."
558                 << "\n5- Search for a student..."
559                 << "\n6- Find the 5 top students..."
560                 << "\n7- Exit.."
561                 << "\nYour option please: ";
562                 cin >> option2;
563
564                 switch (option2)
565                 {
566                 case 1:
567                     system("cls");
568                     display();
569                     system("pause > 0");

```

```

567         goto teacherCase;
568         break;
569     case 2:
570         system("cls");
571         add();
572         system("pause > 0");
573         goto teacherCase;
574         break;
575     case 3:
576         system("cls");
577         delete_student();
578         system("pause > 0");
579         goto teacherCase;
580         break;
581     case 4:
582         system("cls");
583         update_student();
584         system("pause > 0");
585         goto teacherCase;
586         break;
587     case 5:
588         system("cls");
589         search();
590         system("pause > 0");
591         goto teacherCase;
592         break;
593     case 6:
594         system("cls");
595         fiveTop();
596         system("pause > 0");
597         goto teacherCase;
598         break;
599     case 7:
600         cout << "\n\n-----Good Luck, Baye.....:" << endl;
601         goto privilege;
602         break;
603     default:
604         cout << "Ivalid Input....:" << endl;
605         Sleep(1000);
606         goto teacherCase;
607         break;
608     }
609 }
610 else {
611     cout << "Wrong password.....\n\n";
612     trys++;
613     if (trys >= 3)
614     {
615         cout << "You have passed the valid tryies...:\n";
616         goto privilege;
617     }
618     goto password;
619 }
620 break;
621 case 2:
622     studentCase:
623         system("cls");
624         cout << "\n\n-----\n";
625         cout << "                                STUDENT MENUE\n";
626         cout << "-----\n\n";
627         cout << "WELCOME TO THE MENUE WHAT DO YOU WANT TO DO SELECT AN OPTION\n";
628         cout << "PLEASE: "
629             << "\n1- Display all students..."
630             << "\n2- Search for a student..."
631             << "\n3- Find the 5 top students..."
632             << "\n4- Exit.."
633             << "\nYour option please: ";
634         cin >> option2;
635         switch (option2)
636         {
637         case 1:
638             system("cls");
639             display();

```

```

640         system("pause > 0");
641         goto teacherCase;
642         break;
643     case 2:
644         system("cls");
645         search();
646         system("pause > 0");
647         goto teacherCase;
648         break;
649     case 3:
650         system("cls");
651         fiveTop();
652         system("pause > 0");
653         goto teacherCase;
654         break;
655     case 4:
656         cout << "\n\n-----Good Luck, Baye....:" << endl;
657         goto privilege;
658         break;
659     default:
660         cout << "Invalid Input....:" << endl;
661         Sleep(1000);
662         goto studentCase;
663         break;
664     }
665     break;
666 case 3:
667     cout << "\n\n-----Good Luck, Baye....:" << endl;
668     exit(1);
669     break;
670 default:
671     cout << "Invalid Input....:" << endl;
672     Sleep(1000);
673     system("cls");
674     break;
675 }
676
677
678 }
679
680
681 return 0;
682 }

```

